

Project Idea: Family AI Assistant Website -

Family AI Assistant Website Idea: We are building a web-based AI assistant designed to help elderly parents with their daily digital needs while keeping their children connected and informed. Parents can simply talk to the AI in natural language, such as asking for help with a phone problem, online shopping, reminders, or any task they find difficult. The AI will guide them step-by-step and, when necessary, automatically notify their son or daughter about the issue. For example, if a mother wants to buy a product online, the AI can show her suitable options, and if she chooses one, it can send a request to her son to purchase it. The platform can also act as a family communication bridge; if a parent is worried because their child is not answering calls, the AI can provide status updates (with permission), such as informing them that the child is currently driving and is safe. Similarly, children can ask the AI about their parents' well-being and receive important updates. Our vision is to create a smart family companion that makes technology easier for elderly users, reduces dependency on constant phone calls, and strengthens family connections through intelligent, proactive assistance.

Prompt -

Using the project description provided above, create a detailed, production-ready website using the Sarvam AI API as the core AI engine. Design and develop all required features, pages, integrations, and functionality with a modern, responsive, and scalable architecture. Implement the Sarvam AI integration in a way that the API key is not hardcoded. Instead, create a dedicated configuration field, environment variable, or settings section with a clear placeholder such as "PASTE_SARVAM_API_KEY_HERE" so that I can easily copy and paste my Sarvam AI API key later. Ensure that all AI features are fully connected and become operational immediately after the API key is added. After the website is fully completed, perform comprehensive testing, identify all bugs, errors, performance issues, security vulnerabilities, and UI/UX problems, then automatically fix them and ensure the final website is fully optimized, stable, secure, and deployment-ready.

Prompt to Antigravity –

Google Antigravity System Prompt: FamilyBridge AI

Project Directive: You are acting as an expert full-stack developer, UX architect, and QA engineer. Your objective is to build a production-ready, highly accessible web-based application designed to assist elderly parents with digital tasks while keeping their children informed and connected. The platform will use natural language processing to guide parents step-by-step, proactively notify their children of specific needs (like approving a purchase), and act as a secure family communication bridge. Proceed step-by-step to architect, code, and test this platform based on the rigorous requirements below.

1. Technical Architecture & Stack

- Frontend: Next.js (React) using the App Router, Tailwind CSS for styling, and Framer Motion for subtle, accessible micro-animations.
- Backend: Next.js API Routes (Serverless) for standard requests, or a custom Node.js server if real-time features require heavy WebSocket usage.
- Database: PostgreSQL managed via Prisma ORM.
- Real-time Communication: Socket.io or Supabase Realtime for instant family push notifications and status updates.
- AI Engine: Sarvam AI API for natural language processing, intent recognition, and conversational guidance (optimized for Indian languages and contextual English).
- Authentication: NextAuth.js (OAuth/Google for children, and a highly simplified PIN, biometric, or persistent magic-link login for parents).

2. Database Structure (Prisma Schema Guidelines)

- User Model: Stores user ID, full name, role (PARENT or CHILD), family ID, phone number, accessibility preferences, and authentication credentials.
- Family Model: Stores the family group ID, family name, and subscription/tier details.
- InteractionLog Model: Stores historical interaction data, timestamps, AI-detected intent (e.g., SHOPPING, SUPPORT, WELLBEING), and conversational summaries.

- **Task Model:** Stores AI-generated action items (e.g., "Approve purchase for iPad case"), current status (PENDING, COMPLETED), requester ID, and resolver ID.
- **StatusUpdate Model:** Stores real-time context (e.g., "Driving", "At the Doctor"), automatic expiration times, and visibility permission flags.

3. Core Features & Functionality

- **Conversational Interface:** A highly simplified chat interface optimized for voice input utilizing the Web Speech API alongside large, readable text output.
- **Intent Routing:** The AI must intelligently route requests. It must distinguish between tasks requiring step-by-step verbal guides (e.g., "How do I fix my Wi-Fi?") and tasks requiring family intervention (e.g., "I want to buy these shoes").
- **Approval Workflows:** When a parent initiates an online purchase, the AI pauses, drafts a purchase request, displays suitable product cards, and sends a notification to the connected child for final approval and payment.
- **Safety Dashboard:** Children have access to a dashboard displaying their parent's recent interaction frequency, system-generated well-being checks, and pending tasks.
- **Contextual Auto-Replies:** If a parent asks "Where is my son?", the AI securely checks the son's shared status (e.g., "He is currently in a meeting until 3 PM") and responds dynamically without ringing the son's phone.

4. User Flows

- **Parent Flow:** The parent opens the app, taps a massive microphone button, and asks a question. The AI responds with a high-contrast visual guide or creates an action item for the child, asking the parent for verbal confirmation.
- **Child Flow:** The child receives an SMS or push notification, logs into their dense dashboard, reviews the parent's request (e.g., an Amazon link to buy), completes the purchase, and clicks "Resolve." The AI then automatically updates the parent.
- **Status Update Flow:** The child sets their status to "Driving." Later, the parent asks the AI to call them. The AI intercepts, advises the parent that the child is driving, and offers to set a reminder to call back in an hour.

5. UI/UX & Accessibility Requirements (Elderly-First Design)

- **Typography:** Use highly legible fonts like Inter or Roboto with a minimum base size of 18px to 20px.
- **Contrast:** Strictly adhere to WCAG 2.1 AAA contrast ratios. Avoid light grays or low-contrast UI elements.
- **Controls:** Design massive, easy-to-tap buttons with clear, descriptive text labels. Do not use hidden menus, complex hamburger navigations, or ambiguous icons in the parent interface.
- **Feedback:** Provide immediate, clear visual and auditory feedback for every action (e.g., "Recording Started," "Message Sent to Son").
- **Child Interface:** Create a distinct, modern, administrative dashboard UI for the child, allowing for quick triage of notifications, status setting, and task management.

6. AI & External Integrations

- **Sarvam AI Engine:** Integrate Sarvam AI for all conversational logic, translation, and intent extraction.
- **Configuration:** Create a dedicated environment variable config for the AI. In your code, set the API key reference exactly as: `PASTE_SARVAM_AI_API_KEY_HERE`. Ensure the system initializes seamlessly once this key is provided.
- **Fallback Logic:** Implement graceful error handling. If the Sarvam AI API times out, the UI must politely instruct the user to try again in a few moments without crashing.

Link Scraping: Integrate a basic Open Graph fetcher to display beautiful, clickable product cards when the AI suggests items to buy.

7. Security & Privacy

- **Data Encryption:** Ensure all conversational data, tasks, and well-being statuses are encrypted at rest and in transit.
- **Consent Protocol:** The application must explicitly verify permissions before sharing a child's status with a parent or vice versa.
 - **Data Retention:** Build a cron job or backend logic to automatically anonymize or delete routine interaction logs older than 30 days.
- **Rate Limiting:** Protect the backend AI endpoints from abuse using standard IP-based rate limiting.

8. Scalability Considerations

- **Stateless APIs:** Ensure all Next.js API routes are stateless to allow horizontal scaling.
- **Connection Pooling:** Configure Prisma with connection pooling to handle multiple concurrent Database and WebSocket connections without bottlenecks.
- **Caching:** Utilize Next.js built-in caching or Redis to cache static AI responses (like general tech-support guides) to minimize redundant API calls.

9. Execution Plan & QA Automation

- **Phase 1:** Scaffold the application, configure Tailwind, setup NextAuth, and initialize the PostgreSQL database via Prisma.
- **Phase 2:** Build out the distinct Parent and Child user interfaces and routing logic.
- **Phase 3:** Implement the Sarvam AI integration using the `PASTE_SARVAM_AI_API_KEY_HERE` placeholder, wiring up the voice-to-text and text-to-voice loops.
- **Phase 4:** Develop the real-time notification engine, status tracking, and approval workflows.
- **Phase 5 (Testing & QA):** Transition into an automated QA engineer role. Thoroughly test all pages, user journeys, WebSocket connections, API rate limits, and database transactions.
- **Phase 6 (Refinement):** Identify any UI/UX friction points for elderly users. Fix any bugs, broken state management, server errors, or performance issues. You must actively debug and iterate until the entire application is completely functional, optimized, stable, secure, and bug-free.